

Databricks

Complete Syllabus

Azure Data Engineer Edition | 2024-2025

- 01 Databricks Architecture & Workspace
- 02 Delta Lake & Lakehouse
- 03 Apache Spark on Databricks
- 04 Data Engineering with ADF + Databricks
- 05 Databricks SQL & Unity Catalog
- 06 MLflow & Machine Learning
- 07 Structured Streaming & Kafka
- 08 Security, Governance & RBAC
- 09 Workflows & Job Orchestration
- 10 Performance Tuning & Best Practices
- 11 CI/CD & DevOps for Databricks
- 12 Real Interview Questions & Answers

Azure Databricks | Delta Lake | Unity Catalog | MLflow

Table of Contents

FOUNDATION

1. Databricks Architecture & Workspace
2. Delta Lake & Lakehouse Architecture
3. Apache Spark on Databricks

DATA ENGINEERING

4. Data Engineering — ADF + Databricks Pipelines
5. Databricks SQL & Unity Catalog
6. Structured Streaming & Real-Time

ADVANCED

7. MLflow & Machine Learning
8. Security, Governance & RBAC
9. Databricks Workflows & Job Orchestration
10. Performance Tuning & Optimization

PRODUCTION

11. CI/CD & DevOps for Databricks
12. Real Interview Q&A; — Must Know

What is Azure Databricks?

Azure Databricks is a fully managed, cloud-optimized Apache Spark platform built on Microsoft Azure. It provides a collaborative workspace for data engineers, data scientists, and analysts to build data pipelines, train ML models, and run SQL analytics — all on a unified Lakehouse platform.

Component	Description
Control Plane	Managed by Databricks — stores notebooks, job configs, cluster metadata (in Databricks cloud)
Data Plane	Runs in YOUR Azure subscription — VMs (clusters), ADLS storage, VNet
Workspace	Web UI for notebooks, clusters, jobs, SQL warehouses, MLflow
Metastore	Unity Catalog or Hive Metastore — stores table/schema definitions
DBFS	Databricks File System — abstraction over ADLS Gen2 storage

Cluster Types

Type	Use Case	Auto-terminate?	Cost
All-purpose Cluster	Interactive notebooks, development	Optional	Higher (always on)
Job Cluster	Production jobs — created & destroyed per job	Yes (auto)	Lower (ephemeral)
SQL Warehouse	Databricks SQL, BI tools, ad-hoc queries	Yes	Per DBU/query
Single-node Cluster	Small data, testing, ML training locally	Optional	Cheapest

Cluster Configuration

```
# Cluster config (via API or Databricks Asset Bundle YAML)
cluster_config = {
  "cluster_name": "de-prod-cluster",
  "spark_version": "14.3.x-scala2.12",
  "node_type_id": "Standard_DS5_v2", # Azure VM SKU
  "num_workers": 4, # Fixed size
  # OR use autoscaling:
  "autoscale": {
    "min_workers": 2,
    "max_workers": 8
  },
  "spark_conf": {
    "spark.sql.shuffle.partitions": "200",
    "spark.databricks.delta.optimizeWrite.enabled": "true"
  },
  "azure_attributes": {
    "availability": "SPOT_WITH_FALLBACK_AZURE", # Use spot VMs for cost saving
    "spot_bid_max_price": -1
  },
  "init_scripts": [{"workspace": {"destination": "/init/setup.sh"}}],
  "libraries": [{"pypi": {"package": "great-expectations==0.18.0"}}]
}
```

Databricks Runtime (DBR) Versions

Always choose LTS (Long Term Support) versions for production. Key variants: **DBR Standard** (Spark + Python), **DBR ML** (includes TensorFlow, PyTorch, scikit-learn, MLflow), **DBR GPU** (for deep learning on GPU nodes), **DBR Photon** (vectorized C++ query engine for SQL workloads).

Notebooks & Magic Commands

```
# Language per cell – override notebook default language
%python # Python cell
%sql # SQL cell
%scala # Scala cell
%r # R cell
%sh # Shell command (runs on driver node)
%fs # DBFS file system commands
%md # Markdown cell (documentation)
%run # Run another notebook
%pip install great-expectations # Install packages in notebook session

# Example magic commands
%fs ls /mnt/raw/ # List files
%fs cp /mnt/raw/file.csv /tmp/ # Copy file
%sh cat /etc/spark/conf/spark-defaults.conf

# Run another notebook and pass parameters
%run ./utils/etl_helpers $env="prod" $date="2024-01-15"
```

dbutils — Databricks Utilities

```
import dbutils # Auto-available in Databricks notebooks

# File system operations
dbutils.fs.ls("/mnt/raw/")
dbutils.fs.cp("/mnt/raw/data.csv", "/mnt/processed/data.csv")
dbutils.fs.rm("/mnt/temp/", recurse=True)
dbutils.fs.mkdirs("/mnt/output/")

# Secrets – securely retrieve credentials
storage_key = dbutils.secrets.get(scope="my-scope", key="adls-key")
jdbc_password = dbutils.secrets.get(scope="db-scope", key="sql-password")
dbutils.secrets.list("my-scope") # List available secrets

# Widgets – parameterize notebooks (like ADF parameters)
dbutils.widgets.text("env", "dev", "Environment")
dbutils.widgets.dropdown("date", "2024-01-01", ["2024-01-01", "2024-01-15"])
env = dbutils.widgets.get("env")

# Notebook workflow – chain notebooks
result = dbutils.notebook.run("./child_notebook", timeout_seconds=300,
arguments={"env": "prod", "date": "2024-01-15"})
dbutils.notebook.exit("SUCCESS") # Return value to parent

# Display rich output
display(df) # Rich DataFrame viewer
displayHTML("<h1>Hello Databricks</h1>")
```



```
.trigger(availableNow=True) \ # Process all available, then stop  
.start("/mnt/bronze/orders/")
```



```
.format("jdbc") \  
.option("url", "jdbc:sqlserver://server.database.windows.net:1433;database=mydb") \  
.option("dbtable", "(SELECT * FROM orders WHERE updated_at > '2024-01-01') t") \  
.option("user", "admin") \  
.option("password", dbutils.secrets.get("scope", "sql-pwd")) \  
.option("numPartitions", 8) \  
.option("partitionColumn", "id") \  
.option("lowerBound", 1) \  
.option("upperBound", 1000000) \  
.load()
```

ADF + Databricks Integration Patterns

Azure Data Factory (ADF) is the orchestration layer; Azure Databricks is the transformation engine. ADF triggers Databricks notebooks/jobs via the Databricks Linked Service.

Integration Pattern	When to Use	ADF Activity
ADF triggers Databricks Notebook	Simple transformations, quick jobs	Databricks Notebook Activity
ADF triggers Databricks Job	Production pipelines, complex workflows	Databricks Job Activity
ADF Copy + Databricks Transform	Ingest from on-prem/SaaS → transform	Copy Activity + Notebook
Databricks Workflows (standalone)	Databricks-only orchestration, DLT	N/A (Databricks-native)

Delta Live Tables (DLT) — Declarative Pipelines

DLT is Databricks's declarative pipeline framework. You define what tables should contain (not how to compute them) — Databricks handles orchestration, fault tolerance, and monitoring.

[illegible]

```
@dlt.table(
    name="gold_daily_revenue",
    comment="Daily revenue per category"
)

def gold_daily_revenue():
    return (
        dlt.read("silver_orders")
        .groupBy("order_date", "category")
        .agg(
            sum("order_amount").alias("total_revenue"),
            count("order_id").alias("order_count"),
            avg("order_amount").alias("avg_order_value")
        )
    )
```

```
# ■■■ DLT Expectations (Data Quality) ■■■
# @dlt.expect → Record violation in event log (don't drop row)
# @dlt.expect_or_drop → Drop invalid rows
# @dlt.expect_or_fail → Fail pipeline if any violation
# @dlt.expect_all → Multiple rules at once
rules = {
    "valid_amount": "order_amount > 0",
    "valid_status": "order_status IN ('OPEN', 'SHIPPED', 'DELIVERED')",
}
@dlt.table()
@dlt.expect_all_or_drop(rules)
def silver_validated(): ...
```

Databricks SQL

Databricks SQL (DBSQL) is a serverless SQL analytics layer for running SQL queries on Delta Lake tables. It powers BI dashboards, ad-hoc queries, and data exploration — connecting to Power BI, Tableau, and other tools via JDBC/ODBC.

```
-- SQL Warehouse connection string for Power BI / Tableau:
-- Server: adb-xxxx.azure.databricks.net
-- HTTP Path: /sql/1.0/warehouses/<warehouse-id>

-- Create SQL Warehouse (via UI or API)
-- Types: Serverless (recommended), Pro, Classic

-- Databricks SQL — advanced SQL features
-- Window functions, CTEs, pivot, qualify, lateral view

WITH ranked AS (
  SELECT *,
  ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date DESC) AS rn
  FROM catalog.silver.orders
)
SELECT * FROM ranked WHERE rn = 1; -- Latest order per customer

-- QUALIFY clause (Databricks SQL shorthand for window filter)
SELECT customer_id, order_date, order_amount
FROM catalog.silver.orders
QUALIFY ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date DESC) = 1;

-- Dynamic SQL with variable substitution
DECLARE OR REPLACE VARIABLE v_date DATE DEFAULT CURRENT_DATE();
SET VAR v_date = '2024-01-15';
SELECT * FROM catalog.gold.revenue WHERE report_date = v_date;
```

Unity Catalog — 3-Level Namespace

Unity Catalog is Databricks's unified data governance layer. It uses a 3-level namespace: **catalog.schema.table**. It provides centralized access control, data lineage, and auditing across all Databricks workspaces.

```
-- 3-Level namespace: catalog.schema.table
SELECT * FROM prod_catalog.silver.orders;

-- Create catalog / schema
CREATE CATALOG IF NOT EXISTS prod_catalog;
CREATE SCHEMA IF NOT EXISTS prod_catalog.silver;

-- Create managed table (Unity Catalog manages location)
CREATE TABLE prod_catalog.silver.customers (
  customer_id BIGINT NOT NULL,
  name STRING,
  email STRING,
  created_at TIMESTAMP
) USING DELTA;

-- Create external table (you manage location on ADLS)
```

```
CREATE TABLE prod_catalog.silver.orders_ext
USING DELTA
LOCATION 'abfss://silver@storageacct.dfs.core.windows.net/orders/';

-- Grant permissions (fine-grained access control)
GRANT SELECT ON TABLE prod_catalog.silver.orders TO `data-analysts@company.com`;
GRANT MODIFY ON TABLE prod_catalog.silver.orders TO `data-engineers@company.com`;
GRANT ALL PRIVILEGES ON SCHEMA prod_catalog.silver TO `de-team@company.com`;

-- Row-level security with row filter
CREATE FUNCTION region_filter(region STRING)
RETURNS BOOLEAN
RETURN is_member('emea-team') OR region = 'GLOBAL';

ALTER TABLE prod_catalog.silver.orders
SET ROW FILTER region_filter ON (region);

-- Column masking (PII protection)
CREATE FUNCTION mask_email(email STRING)
RETURNS STRING
RETURN CASE WHEN is_member('pii-viewers') THEN email
ELSE CONCAT(LEFT(email,2), '***@***.com') END;

ALTER TABLE prod_catalog.silver.customers
ALTER COLUMN email SET MASK mask_email;

-- Data lineage – view in Unity Catalog UI automatically
-- Column-level lineage tracked automatically in DLT and SQL

-- List catalogs / schemas / tables
SHOW CATALOGS;
SHOW SCHEMAS IN prod_catalog;
SHOW TABLES IN prod_catalog.silver;
DESCRIBE DETAIL prod_catalog.silver.orders;
```

[illegible]

```

).whenMatchedUpdateAll() \
.whenNotMatchedInsertAll() \
.execute()

```

```
query2 = parsed.writeStream \
    .foreachBatch(upsert_to_delta) \
    .option("checkpointLocation", "/mnt/checkpoint/upsert/") \
    .trigger(processingTime="1 minute") \
    .start()
```

```
# Trigger options
# .trigger(processingTime="1 minute") # Fixed interval micro-batch
# .trigger(once=True) # Run once (deprecated)
# .trigger(availableNow=True) # Process all available, then stop
# .trigger(continuous="1 second") # Near real-time (experimental)
```

MLflow — End-to-End ML Lifecycle

MLflow is open-source and built into Databricks. It tracks experiments, packages models, manages the model registry, and serves models. Every Databricks workspace has a managed MLflow server.

MLflow Component	Purpose
Tracking	Log parameters, metrics, artifacts per experiment run
Models	Package model with dependencies (MLmodel format)
Model Registry	Version control for models — Staging / Production / Archived
Projects	Reproducible ML code packages (MLproject file)
Model Serving	REST endpoint for real-time inference (Databricks Model Serving)

[illegible]

[illegible]


```
spark.sql("GRANT MODIFY ON TABLE prod_catalog.silver.orders TO `de-team`")

# Revoke
spark.sql("REVOKE SELECT ON TABLE prod_catalog.silver.pii_data FROM `contractors`")

# Check grants
spark.sql("SHOW GRANTS ON TABLE prod_catalog.silver.orders").show()
```


[illegible]

[illegible]

[illegible]

[illegible]

12 Real Interview Q&A — Must Know

Q1: What is the difference between All-Purpose and Job clusters?

All-Purpose clusters stay running for interactive development — expensive but immediately available. Job clusters are created fresh for each job run and terminated automatically — cheaper for production. Always use Job clusters in production to minimize cost.

Q2: Explain the Medallion Architecture (Bronze/Silver/Gold).

Bronze = raw ingested data as-is (no transformation, append-only Delta). Silver = cleansed, validated, deduplicated data (typed, filtered). Gold = business-aggregated data for BI/reporting. Each layer adds quality. Delta Lake enables ACID + time travel at every layer.

Q3: What is Delta Live Tables (DLT) and when would you use it?

DLT is Databricks's declarative pipeline framework. You define tables with `@dlt.table` and data quality with `@dlt.expect` — Databricks handles orchestration, retry, and monitoring automatically. Use DLT for streaming + batch unified pipelines, auto schema evolution, and built-in data quality rules.

Q4: How does Unity Catalog differ from the Hive Metastore?

Hive Metastore is workspace-scoped (one metastore per workspace). Unity Catalog is account-scoped — a single metastore shared across all workspaces. Unity Catalog adds: 3-level namespace (catalog.schema.table), fine-grained RBAC, column masking, row filters, data lineage, and audit logging.

Q5: How do you handle late-arriving data in streaming?

Use `.withWatermark('event_time', '10 minutes')` to define the late data tolerance. Spark drops events older than the watermark threshold. For batch, use `'replaceWhere'` in Delta Lake to reprocess only affected partitions without rewriting the entire table.

Q6: How would you optimize a slow Spark job?

Step 1: Run `.explain()` to find bottlenecks (shuffles, full table scans). Step 2: Enable AQE (`spark.sql.adaptive.enabled=true`). Step 3: Broadcast small tables. Step 4: Run `OPTIMIZE + ZORDER` on Delta tables. Step 5: Check for data skew and apply salting. Step 6: Tune shuffle partitions to match data size.

Q7: What is Liquid Clustering in Delta Lake?

Liquid Clustering (DBR 13.3+) is a self-tuning alternative to static partitioning + `ZORDER`. You define `CLUSTER BY` columns once; Databricks automatically reorganizes data as it grows. No need to choose partition granularity — it adapts dynamically and avoids small file problems.

Q8: How do you implement incremental load in Databricks?

Use `MERGE (upsert)` on Delta Lake with a watermark column (`updated_at`). Load only records where `updated_at > last_load_timestamp` (stored in a control table). `MERGE` handles inserts and updates atomically. For streaming, use Auto Loader with checkpointing.

Q9: What is the difference between `cache()` and Delta cache?

`cache()` / `persist()` = Spark cache — stores `DataFrame` in executor memory/disk as JVM objects. Cleared when cluster restarts. Delta cache (Databricks-native) = caches Parquet/Delta file blocks on local SSD of worker nodes. Faster than Spark cache, persists across jobs, enabled by default on Databricks.

Q10: How does Databricks integrate with Azure DevOps for CI/CD?

Use Databricks Asset Bundles (DAB) to define jobs/pipelines as YAML in Git. Azure DevOps pipeline: (1) Unit test with `pytest` + local Spark, (2) Bundle deploy to dev, (3) Integration test run, (4) Promote to prod on merge to main. Secrets stored in Azure Key Vault, accessed via Databricks secret scopes.

■ **Key Takeaway:** For Azure Data Engineer interviews, master these 5 topics: (1) Medallion Architecture with Delta Lake MERGE, (2) Unity Catalog RBAC + 3-level namespace, (3) Delta Live Tables for declarative pipelines, (4) AQE + Photon + ZORDER for performance, (5) Databricks Asset Bundles for CI/CD. Always be ready to design an end-to-end pipeline from raw ADLS data to a Gold BI layer.